

CS780 Capstone Project: OBELIX: the Warehouse Robot

Aryan Chaturvedi

230212

`aryanc23iitk.ac.in`

Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This report presents the design, training, and evaluation of RL agents for the OBELIX warehouse robot task. The objective is to locate a grey box using 18-bit sensor inputs, attach to it, and push it to the arena boundary under a Partially Observable Markov Decision Process (POMDP). We explore tabular Double Q-learning, Dueling Double DQN with prioritized replay, D3QN-PER with n-step error, and PPO with frame stacking. Our results show that observation-driven reward shaping, phase-aware training, and structured exploration are more helpful than using complex algorithms. Final agent performance demonstrates that we need robust training and better reward shaping.

1 Competition Result

Codabench Username: a_230212

Final leaderboard rank (Test Phase): 65

Leaderboard rank – Level 1 (static): 85

Leaderboard rank – Level 2 (blinking): 81

Leaderboard rank – Level 3 (moving + blinking): 34

Number of valid submissions made (Development(Level 1 , Level 2, Level 3) and Test Phase):

Phase	Number of Submissions
Development (Level 1)	5
Development (Level 2)	2
Development (Level 3)	3
Test Phase	2

Table 1: Number of valid submissions across different phases

2 Introduction and Problem Description

2.1 Motivation

Real-world robotic systems must operate under limited and noisy sensor information, making decision-making inherently uncertain. The OBELIX environment helped to study how agents can learn reliable multi-step behaviors such as navigation and object manipulation under partial observability.

2.2 Task Description

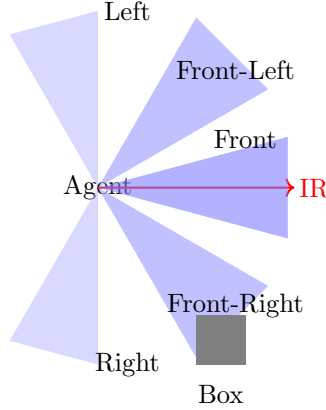
Agent’s objective is to complete a sequential task involving a grey box. The task can be broken down into four stages: *find* $\rightarrow$ *attach* $\rightarrow$ *push* $\rightarrow$ *unwedge*. First, the agent must explore the environment to locate the box. Upon making contact, it attaches to the box. Once attached, the agent must push the box towards the arena boundary. In some cases, the box may get stuck near walls or obstacles, requiring the agent to adjust its direction and successfully unwedge it before completing the task.

2.2.1 Environment and Observations

The environment is a 2D arena where the agent interacts using an 18 bits vector - 2^{18} states. This includes directional sonar sensors, an infrared sensor (proximity detection), and a “stuck” indicator that signals collisions with walls or boundaries.

Example Observation: $\underbrace{0, 1, 0, 0}_{\text{Left}} \mid \underbrace{1, 0, 0, 1}_{\text{Front}} \mid \underbrace{1, 0, 0, 0}_{\text{Right}} \mid \underbrace{0, 0, 0, 0}_{\text{Rear}} \mid \underbrace{0}_{\text{IR}} \mid \underbrace{0}_{\text{Stuck}}$

2.2.2 Sensor Visualization



2.3 Difficulty Levels and Variations

Level	Setting	Key Challenge
Level 1	Box remains stationary	We need efficient exploration under sparse rewards
Level 2	Box disappears and reappears	Perceptual uncertainty and requires robustness to missing observations
Level 3	Box moves along a random trajectory while blinking	requires real-time tracking.
Wall Obstacle: Barriers in the arena, forces agent to learn navigation and obstacle avoidance		

Table 2: Difficulty levels and their associated challenges

2.4 POMDP Nature and Challenges

- The task is a POMDP as the agent observes only partial state information.
- Identical observations can represent different states (e.g., searching vs. attached).
- Sparse rewards and delayed feedback make learning difficult.
- Attachment state is not directly observable and must be inferred.

2.5 Objective

The primary goal of this work is to design an RL agent that maximizes the mean cumulative reward across all difficulty levels under partial observability and environmental variations(wall obstacles). Importantly, we do not only focus on maximizing reward, but also on ensuring that the agent does not converge to sub-optimal policy(spining here)

3 Approach and Model Evolution

3.1 Starter Code: DDQN

We started with a standard Double DQN using raw 18-bit observations and ϵ -greedy exploration.

Issue: The agent performed poorly (≈ -17000), learning to rotate in place to avoid wall penalties instead of exploring. Sparse rewards prevented it from discovering the box.

Insight: Naive RL fails due to reward sparsity and lack of meaningful exploration.

→ Solution: Need to improve learning signal and sample efficiency(to train in lesser samples).

3.2 D3QN + PER

We upgraded to Dueling Double DQN with Prioritized Experience Replay to emphasize important transitions. Main objective was to improve on learning signal, so agent can explore more.

Issue: Although scores improved (≈ -1900), the agent still converged to a rotation-heavy policy, avoiding risky forward movement.

Insight: The agent is optimizing a *safe but useless policy* (local optimum).

→ Solution: Need to improve credit assignment(so that agent moves forward) and guide exploration .

3.3 D3QN-PER with N-Step returns + Reward Shaping

We introduced n-step returns and sensor-based reward shaping to propagate rewards and guide movement toward the box.

Issue: The agent exploited reward shaping by rotating to activate sensors, reinforcing the rotation trap.

Insight: Reward shaping must be *action-aware*; otherwise, it leads to unintended exploitation.

→ Solution: We shift towards a more structured and scalable approach using deep RL with improved exploration and training efficiency.

3.4 D3QN + PER + N-step + Parallel Processing

We extended the D3QN-PER with n-step error and parallel environment training to improve both sample efficiency and learning stability by running more episodes on multiple cores of CPU.

Issue: Despite these improvements, the agent still exhibited suboptimal behaviors such as rotation bias, especially in complex scenarios involving blinking and moving boxes.

Insight: While architectural improvements enhanced learning speed, they were insufficient to overcome challenges arising from partial observability and delayed rewards.

→ Solution: We should try policy-based methods to account to complexity of (blinking + moving) box and improve exploration dynamics.

3.5 PPO

We implemented PPO with entropy-based exploration to encourage diverse behaviors.

Issue: Due to sparse rewards, PPO failed to learn meaningful policies, as early trajectories lacked positive signals.

Insight: On-policy methods require frequent reward signals, which are absent in this environment.

→ *Solution:* Need to improve observability and sample efficiency as it is not converging in 10000 ep.

3.6 PPO with Frame Stacking and Parallel Environments

We enhanced PPO using frame stacking (temporal context) and parallel environments (faster episodes running by exploiting multiple cores).

Outcome: Learning improved, with better exploration and smoother training curves, but convergence remained slow due to reward sparsity.

Final Insight: The main challenge is not algorithm choice but handling partial observability and sparse rewards effectively.

4 Experiments and Implementation Details

4.1 Training Setup

All single-environment experiments were conducted on CPU (Intel Core series). Parallel-environment experiments for D3QN and PPO were run on multiple cores of CPU(22 cores).

4.2 Hyperparameter Table

Table 3: Hyperparameters used for different model variants

Parameter	D3QN-PER with n-step	D3QN-PER with n-step + parallel	PPO
Episodes	1,000	10,000	60,000
Max steps/episode	1,000	5,000	5,000
Gamma (γ)	0.997	0.995	0.99
Learning rate	5×10^{-4}	5×10^{-4}	3×10^{-4}
Batch size	128	64	64
Replay buffer	100k	100k	N/A (on-policy)
Warmup steps	5,000	5,000	N/A
Epsilon start/end	1.0 / 0.05	1.0 / 0.1	N/A
Epsilon decay steps	100k	150k	N/A
PER α	0.6	0.4	N/A
N-step	1	4	N/A (GAE $\lambda = 0.95$)
Soft update τ	0.005	0.005	N/A
Reward clipping	[-200, 200]	[-50, 50]	None
Frame stack	1	1	4
Optimizer	Adam	Adam	Adam
Target update	Soft ($\tau = 0.005$)	Soft ($\tau = 0.005$)	N/A
Parallel envs	1	1	22
Observation dim	18	18	72

Continued on next page

Parameter	D3QN	D3QN-PER with n-step	PPO
Hardware	CPU	CPU/GPU	GPU/MPS
Training time	45–90 min	22 hours	~ 6 hours
Actions	5	5	5

All models were trained under identical environment settings to ensure fair comparison. Typical training consisted of 3,000 episodes with 1,000 steps per episode ($\approx$ 3M environment steps), taking approximately 45–90 minutes on CPU for DQN-based methods. PPO with 22 parallel workers achieved similar experience in approximately 6 hours.

4.3 Evaluation Strategy

Local evaluation was performed using 5 random seeds (offsets: 0, 100, 200, 300, 400) to measure variance. We tracked two metrics:

- (1) Mean cumulative episode return with wall, (2) Mean cumulative episode return without wall

Learning curves were smoothed using a 50-episode moving average. To avoid overfitting to fixed seeds, each episode used a varying seed (base + episode number), ensuring diverse environment configurations.

5 Results

5.1 Performance on Development Phases

Model	Level 1 (Static)	Level 2 (Blinking)	Level 3 (Moving)
D3QN	-17000	-	-
D3QN + PER + N-step	-1990.3	-	-
PPO	-	-	-1514.94
PPO + Frame Stack	-1828.4	-1828.4	-1828.4

Table 4: Performance of different models on development phases

5.2 Final Test Phase Results

Model	Level 1	Level 2	Level 3
Best Submitted Model	-1828.4	-1828.4	-1828.4

Table 5: Final performance on test phase across all difficulty levels

5.3 Learning Curves

We analyze the training dynamics of different models using learning curves (episode vs. cumulative reward). Early-stage models such as DDQN and D3QN show flat or unstable curves due to sparse rewards and suboptimal policies.

In contrast, improved variants demonstrate gradual reward improvement, indicating better exploration and learning stability. Frame stacking and parallel environments further smooth the learning curves by providing richer and more diverse training signals.

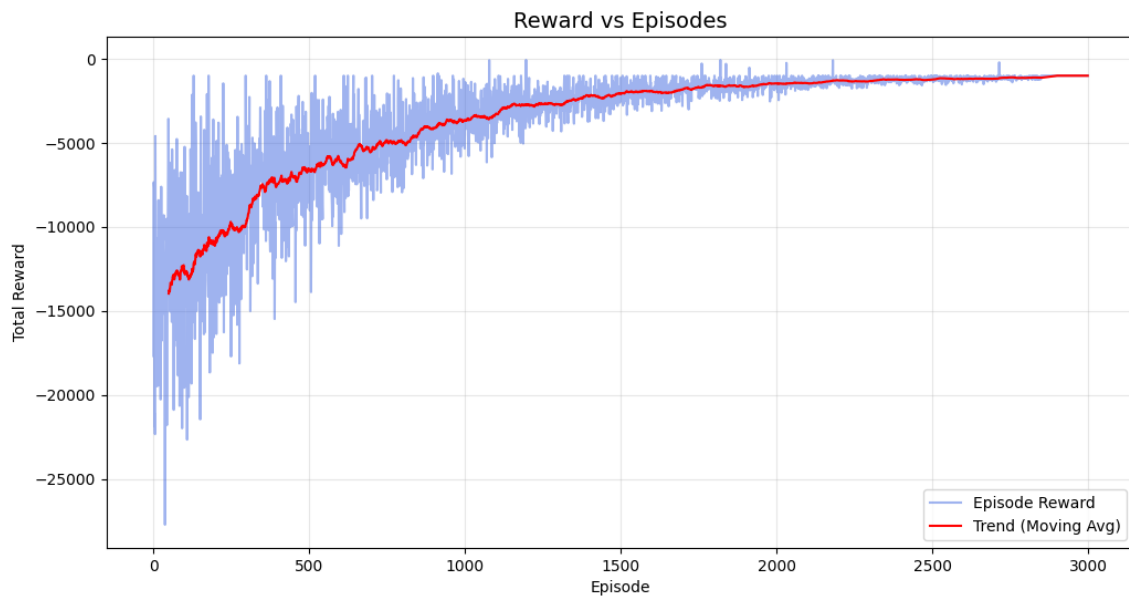


Figure 1: Learning curves for D3QN-PER with n-step return

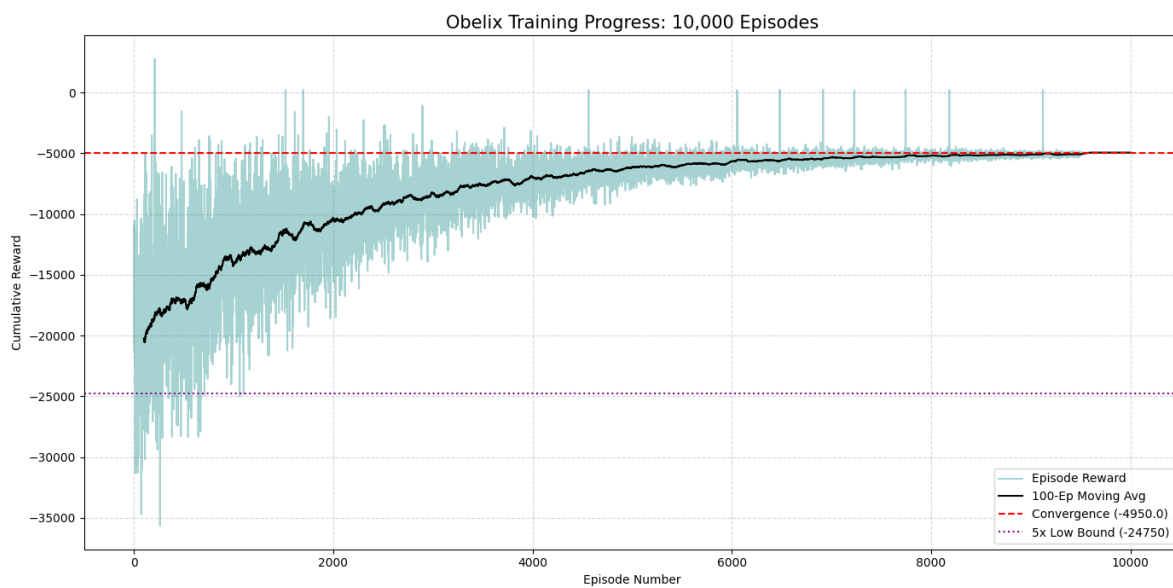


Figure 2: Learning curves for D3QN-PER with n-step return and parallel processing

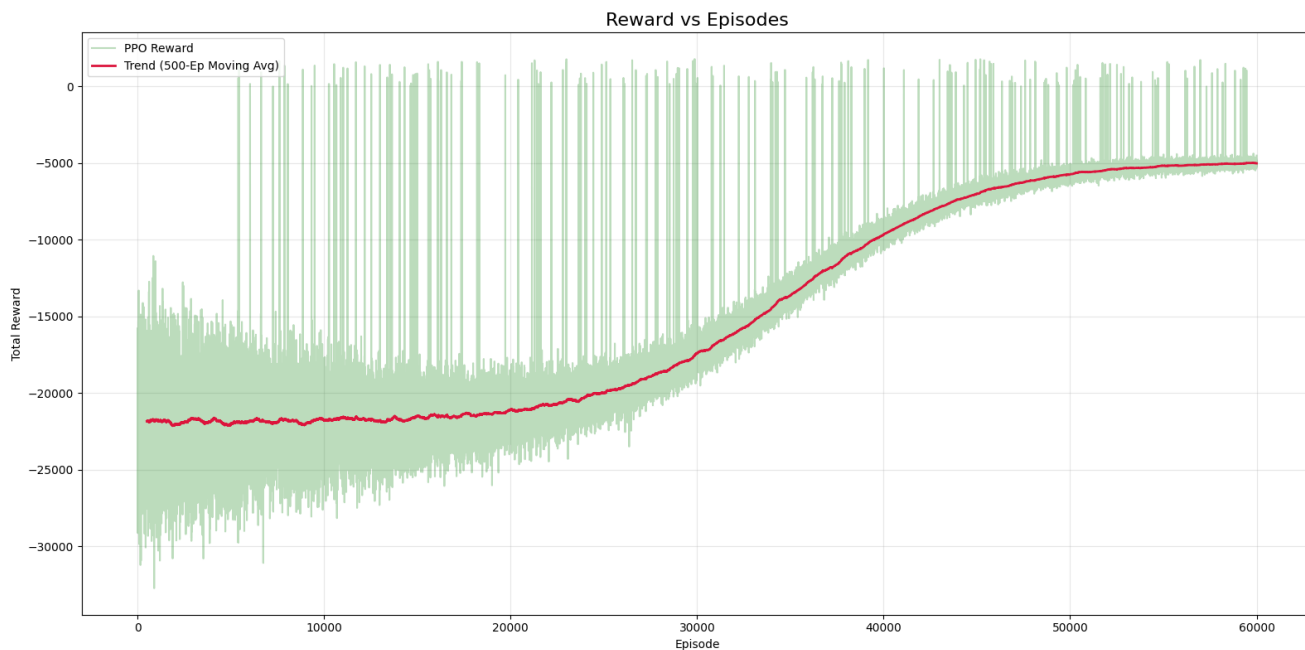


Figure 3: Learning curves for PPO with frame-stacking and parallel processing

5.4 Best Model Analysis

Among all approaches, the best-performing model was (**PPO + Frame Stack + parallel processing**).

Reasons for superior performance:

- **Improved sample efficiency:** Leveraged prioritized sampling or parallel environments to learn faster.

Overall, the objective of optimal policy couldn't be satisfied, leading to sub-optimal policy of spinning agent.

6 Error Analysis and Discussion

6.1 Level 1 Failures

Even when the box is stationary, the agent does not always succeed. One common issue is poor initial positioning. If the agent starts facing away from the box and the box is outside its sensor range, it has to explore randomly, which can take a large number of steps. Another issue occurs after attachment: the agent sometimes pushes the box into a corner where it gets stuck against a wall. Since the box has not reached the boundary, the episode does not end, but no further progress is possible.

6.2 Level 2 Failures

Blinking makes the problem harder because the box temporarily disappears. When this happens during movement, the agent loses all sensor signals related to the box and cannot maintain its direction. As a result, it often starts rotating or re-exploring, effectively restarting the task. While frame stacking helps to some extent, models without memory struggle significantly in this setting.

6.3 Level 3 Failures

When the box both moves and blinks, the task becomes much more challenging. The agent finds it difficult to predict the box's motion, especially because the direction can change randomly. In many

cases, the agent approaches the box but overshoots due to its higher speed, and then loses track of it when it changes direction. This leads to repeated failed attempts.

6.4 Wall Obstacle Failures

The wall obstacle introduces an additional navigation challenge. The agent often collides with the edges of the wall while trying to pass through the gap, resulting in frequent penalties, leads to spinning (sub-optimal policy). This becomes even harder when the box is on the other side of the wall.

6.5 Limitations and Improvements

Our approach still has many limitations. The agent does not have explicit memory, so it struggles in situations where information is temporarily missing (e.g., blinking). The reward function also requires careful tuning and may not generalize well to different setups. With more time, we would explore adding memory (e.g., LSTM), improving reward design, and using curriculum learning to gradually increase difficulty.

6.6 Surprising Findings

One interesting observation was that tabular methods performed more consistently in the early stages compared to deep learning models. This is likely because they update each state independently and do not suffer from generalization issues. We also noticed that the stuck penalty sometimes affects correct behavior near walls, suggesting that the reward design could be further improved.

7 Conclusion

In this project, we found that the main challenge in the OBELIX task is not choosing a complex algorithm, but designing the right reward signals and handling partial observability. Even advanced methods like D3QN-PER and PPO struggle if the reward structure does not guide the agent properly.

The key takeaway is that in POMDP setting, good reward design and proper handling of uncertainty are more important than increasing model complexity. Techniques such as reward shaping, better exploration strategies, and using temporal information play a crucial role in learning effective policies.

In the future, this work can be extended by adding memory-based models (such as LSTMs), using curriculum learning to train the agent step by step, and exploring more suitable RL methods. It would also be interesting to apply these ideas to real-world robotic systems with more realistic sensors and environments.

8 Related Links

GitHub Repository:

CS780 Project Repository

YouTube Demo Video:

Project Demonstration Video

LLM Usage Declaration

- Which Large Language Model (LLM) did you use to complete the competition or write the report?
Gemini

- In which part did you use the LLM, and how did it help you?

Part	Used LLM? (Yes/No)	Purpose of Using LLM
Understanding concepts	Yes	Clarifying POMDP formulation and sensor-related doubts
Ideation / brainstorming	Yes	Discussing reward shaping strategies and phase separation ideas
Debugging small code snippets	Yes	Fixing n-step buffer edge cases and PER SumTree implementation
Plotting / visualization code	No	
Grammar / writing style	Yes	Proofreading and improving report clarity
Report structure / section ideas	No	Followed the provided template strictly
Other (specify)	No	Algorithm implementation done independently

- Did you refer to any other sources or websites apart from the LLM and lecture slides?

If yes, please mention clearly (papers, blogs, GitHub repos, textbooks etc.)

Schaul et al. (2015) Prioritized Experience Replay; Wang et al. (2016) Dueling Network Architectures for Deep Reinforcement Learning; Schulman et al. (2017) Proximal Policy Optimization Algorithms; Mahadevan & Connell (1992) Automatic Programming of Behaviour-based Robots using Reinforcement Learning; OpenAI Spinning Up documentation

- **Personal contribution:** All environment exploration, debugging of observations (sensor bit, stuck-flag edge cases, attachment detection heuristic), design and implementation of phase-separation reward shaping, frame stacking PPO architecture, hyperparameter tuning, and experimental analysis were carried out independently. LLM assistance was limited to conceptual clarification and proofreading, and was not used for core algorithmic or analytical contributions.

References

- [1] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” *arXiv preprint arXiv:1511.05952*, 2015.
- [2] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, “Dueling network architectures for deep reinforcement learning,” *International Conference on Machine Learning (ICML)*, 2016.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.

- [4] S. Mahadevan and J. Connell, “Automatic programming of behaviour-based robots using reinforcement learning,” *Artificial Intelligence*, vol. 55, no. 2–3, pp. 311–365, 1992.